

第二章 STL源码思想（面试突击★★★★★）

前言

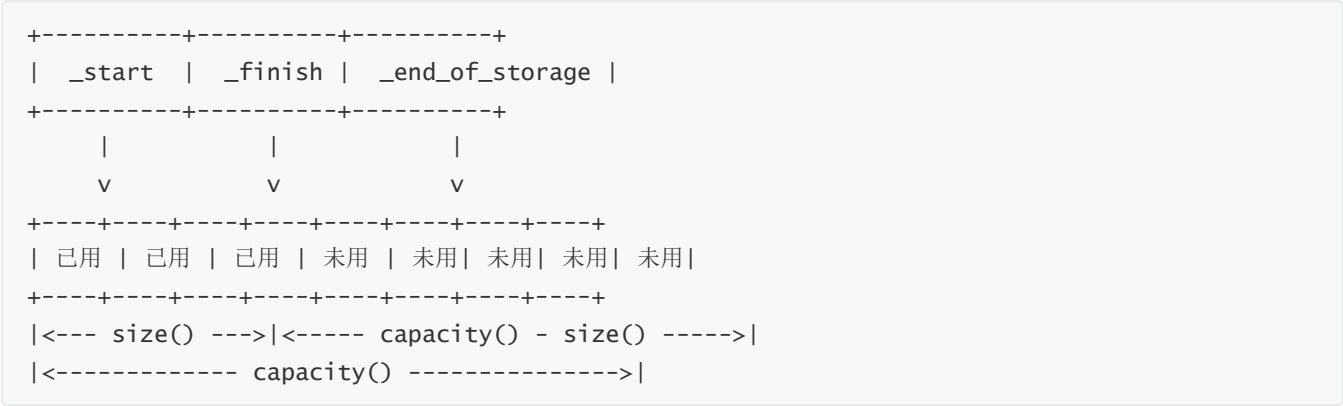
STL（Standard Template Library）是C++的核心库，也是面试中必问的重灾区。本章从源码角度深入剖析STL六大组件的设计思想与实现细节，帮助读者在面试中从容应对。

STL六大组件：容器（Containers）、算法（Algorithms）、迭代器（Iterators）、仿函数（Functors）、适配器（Adapters）、分配器（Allocators）。本章聚焦容器，因为它是面试中最常被问到的部分。

1. vector

1.1 底层数据结构

vector底层是一个连续内存的动态数组。使用三个指针管理内存：



- `_start` 指向数组首元素
- `_finish` 指向最后一个元素的下一个位置
- `_end_of_storage` 指向分配内存的末尾

1.2 时间复杂度

操作	时间复杂度	说明
随机访问 <code>v[i]</code>	$O(1)$	连续内存，直接偏移计算
<code>push_back</code> (未扩容)	$O(1)$	尾部插入
<code>push_back</code> (扩容时)	$O(n)$	需要重新分配+拷贝；均摊 $O(1)$
<code>pop_back</code>	$O(1)$	尾部删除，仅移动 <code>_finish</code> 指针
<code>insert</code> (中间)	$O(n)$	需要移动后续所有元素
<code>erase</code> (中间)	$O(n)$	需要移动后续所有元素
<code>insert/erase</code> (尾部)	$O(1)$	实际等同于 <code>push_back/pop_back</code>

操作	时间复杂度	说明
查找（无序）	$O(n)$	线性遍历
查找（有序+二分）	$O(\log n)$	使用lower_bound等

1.3 空间复杂度

- 空间占用: `element_size * capacity() + 3 * sizeof(pointer)`
- 平均会有 `capacity() - size()` 的未使用空间
- 最优情况: `size() == capacity()`, 无浪费
- 最差情况: 刚扩容后 `capacity() == 2 * size()` (如果用2倍扩容), 浪费50%

1.4 扩容机制

GCC libstdc++ 采用2倍扩容

```
// GCC libstdc++ vector扩容核心逻辑（简化）
// 源码位置: bits/stl_vector.h, bits/vector.tcc

template<typename T>
void vector<T>::_M_realloc_insert(iterator __position, const T& __x) {
    const size_type __len =
        _M_check_len(1, "vector::_M_realloc_insert"); // 计算新容量
    // ...
}

size_type _M_check_len(size_type __n, const char* __s) const {
    if (max_size() - size() < __n)
        __throw_length_error(__s);
    const size_type __len = size() + std::max(size(), __n);
    // 如果当前不为空: __len = size() + max(size(), 1) = 2 * size()
    // 如果当前为空: __len = 0 + max(0, 1) = 1
    return (__len < size() || __len > max_size()) ? max_size() : __len;
}
```

MSVC 采用1.5倍扩容

```
// MSVC STL vector扩容核心逻辑（简化）
size_type _Calculate_growth(const size_type _Newsize) const {
    const size_type _Oldcapacity = capacity();
    if (_Oldcapacity > max_size() - _Oldcapacity / 2) {
        return _Newsize;
    }
    const size_type _Geometric = _Oldcapacity + _Oldcapacity / 2; // 1.5倍
    if (_Geometric < _Newsize) {
        return _Newsize;
    }
    return _Geometric;
}
```

为什么选择1.5倍 vs 2倍扩容？

1.5倍扩容的优势：内存可复用

2倍扩容的内存使用模式（以初始容量1为例）：
1, 2, 4, 8, 16, 32, 64, 128, ...

释放的内存大小: 1, 2, 4, 8, 16, 32, 64...
当前需要的内存: 2, 4, 8, 16, 32, 64, 128...

释放的总和 = $1+2+4+...+64 = 127 < 128$ （当前需要的）
说明：之前释放的所有内存加起来，永远小于下一次需要的内存
==> 内存永远无法被后续分配复用，只能向OS申请新页

1.5倍扩容的内存使用模式（以初始容量1为例）：
1, 2, 3, 4, 6, 9, 13, 19, 28, 42, 63, 94, ...

释放的内存大小(累计): 1, 3, 6, 10, 16, 25, 38, 57, 85...
当前需要的内存: 2, 3, 4, 6, 9, 13, 19, 28, 42...

在某个时刻，释放总和会超过当前需要的，之前释放的内存可以被复用！

为什么不是1.5倍而是k=1.5？

数学上，扩容因子k满足：当 $1/(k-1) > 1$ 即 $k < 2$ 时，释放的内存最终可以被复用。

- k = 2：释放的总和永远 < 当前需要，无法复用
- k = 1.5：释放的总和会超过当前需要，可以复用
- k = 1.1：扩容太频繁，导致频繁的重新分配

1.5是一个工程上的平衡点。

为什么不是1.1倍？

扩容因子	从1扩容到1000需要的次数	realloc次数	优缺点
2.0	$\text{ceil}(\log_2(1000)) = 10$	少	内存无法复用，浪费大
1.5	$\text{ceil}(\log_{1.5}(1000)) = 18$	中	内存可复用，平衡
1.1	$\text{ceil}(\log_{1.1}(1000)) = 73$	多	复制开销巨大，不可接受

扩容时的元素迁移

C++11之前用拷贝，C++11起用移动语义：

```
// 如果T的移动构造函数是noexcept的
// vector在扩容时会使用move而非copy
// 原因：如果移动过程中抛异常，已移动的元素无法回滚

// 检查方法（GCC源码中的type trait）：
static_assert(std::is_nothrow_move_constructible<T>::value,
              "Element type should have noexcept move constructor");
```

如果 `move_if_noexcept` 返回false（即移动构造器非noexcept），则退回使用拷贝构造，保证异常安全。

1.5 内存布局

栈/堆上vector对象：

```
+-----+-----+-----+
| _start | _finish | _end_of |
| 0x1000 | 0x100C | _storage|
|       |       | 0x1014 |
+-----+-----+-----+
      |           |           |
      v           v           v
```

堆上的实际数据：

```
地址： 0x1000  0x1004  0x1008  0x100C  0x1010  0x1014
      +-----+-----+-----+-----+-----+-----+
      |  [0]  |  [1]  |  [2]  | 未使用 | 未使用 | 未使用 |
      +-----+-----+-----+-----+-----+-----+
      size() = 3      |      capacity() - size() = 3
                      |
                      |
                      +-----+
                      | capacity() = 6
```

1.6 设计理由：为什么用连续内存？

设计选择	理由
连续内存	提供O(1)随机访问；利用CPU cache locality
动态扩容	编译时不需要知道大小；运行时灵活调整
尾部操作优化	80%的场景只需要尾部追加/删除
三指针管理	分离size和capacity，减少不必要的重新分配

为什么不用链表做动态数组？

链表随机访问O(n)，vector是O(1)。对于需要随机访问的场景（如二分查找），链表性能差距巨大。且链表每个节点有额外指针开销。

1.7 源码思想

```
// 源码核心设计思想（GCC libstdc++简化版）
template<typename _Tp, typename _Alloc = std::allocator<_Tp>>
class vector {
    // 核心思想1：用三个指针而非size_t维护状态
```

```

pointer _M_start;           // 指向第一个元素
pointer _M_finish;          // 指向最后一个元素+1
pointer _M_end_of_storage; // 指向内存尾部+1

// 核心思想2: size()通过指针差值计算, 不需要单独存储
size_type size() const {
    return size_type(_M_finish - _M_start);
}

// 核心思想3: 通过traits在编译期选择最优策略
// _GLIBCXX_MOVE3: 如果类型支持noexcept_move则移动, 否则拷贝
// std::copy vs std::uninitialized_copy
// std::move vs std::copy (对POD类型, move == copy)
};

```

异常安全保证级别:

- 强异常安全: `push_back`、`insert` (元素有noexcept移动时)
- 基本异常安全: `erase`、`pop_back`

1.8 实际应用

场景	说明
动态列表	元素数量在运行时确定且频繁尾部操作
图像缓冲	像素数据需要连续内存以传给图形API
排序场景	需要随机访问迭代器
缓存IO	连续内存适合做缓冲区和批量读写

1.9 面试题

Q1: vector的push_back是线程安全的吗? 为什么?

A: 不是。C++标准库不做线程安全保证。多个线程同时调用push_back会导致数据竞争(修改_finish指针和写元素)。需要使用mutex保护或者使用线程安全的容器。

Q2: vector扩容时迭代器会失效吗?

A: 会。扩容会导致所有迭代器、指针、引用全部失效(因为内存重新分配)。如果未发生扩容, 只有插入位置之后的迭代器失效。

Q3: 如何减少vector的扩容次数?

A: 使用reserve()预分配足够容量。如果已知大致元素数量, 调用reserve(n)可以避免多次扩容带来的拷贝开销。

Q4: vector有什么特殊之处?

A: vector是特化版本, 不存储bool而是存储bit。每个元素占1位而非1字节。它不能返回bool&引用, 返回的是代理对象`vector<bool>::reference`。这破坏了容器的基本约定, 被认为是设计缺陷。

Q5: size()和capacity()的区别?

A: size()返回当前元素数量, capacity()返回当前已分配内存能容纳的元素数量。capacity() >= size()。当size() == capacity()时push_back触发扩容。

Q6: clear()后会释放内存吗?

A: 不会。clear()只销毁元素并把size置为0，capacity()不变（内存保留不还给OS）。要释放内存，用 `vector<int>().swap(v)` 或C++11的 `shrink_to_fit()`。

Q7: vector和array的区别?

A: vector是动态数组（运行时可变大小），array是静态数组（编译期固定大小）。array在栈上分配，vector数据在堆上。array没有size/capacity开销。

Q8: emplace_back vs push_back的区别?

A: emplace_back通过完美转发直接在容器内构造对象，避免了临时对象的构造+移动；push_back先构造临时对象再移动/拷贝到容器内。对于复杂对象，emplace_back更高效。

```
// emplace_back vs push_back
vector<pair<int, string>> v;
v.push_back(make_pair(1, "hello")); // 构造临时pair -> 移动到vector -> 销毁临时
v.emplace_back(1, "hello");         // 直接在vector内部构造pair
```

Q9: 当vector存储的是指针时，clear()会释放指针指向的内存吗?

A: 不会。clear()只销毁元素本身（即指针），不释放指针所指向的内存。需要手动释放每个指针指向的内存后再clear。

Q10: 为什么使用移动语义对vector扩容很重要?

A: 传统扩容：分配新内存 -> 拷贝所有旧元素 -> 销毁旧元素 -> 释放旧内存。移动语义下：分配新内存 -> 移动所有旧元素 -> 不需要销毁 -> 释放旧内存。移动远比拷贝高效（如string只需复制指针不复制内容）。条件是移动构造必须是noexcept的。

Q11: vector在哪些操作后引用/指针会失效?

A:

- 扩容操作（push_back可能触发）：所有指针/引用/迭代器失效
- insert/emplace：插入位置之后的迭代器失效（未扩容时）
- erase：被删除元素及之后所有元素的迭代器失效
- pop_back：仅被删除元素的迭代器失效
- clear：所有迭代器失效

Q12: 为什么要区分size()和capacity()?

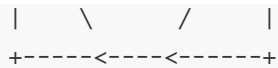
A: 解耦"逻辑大小"和"物理内存"。如果每次push_back都重新分配内存，性能不可接受（ $O(n^2)$ ）。预分配更多内存，每次扩容均摊操作为 $O(1)$ 。这是典型的"用空间换时间"设计。

2. list

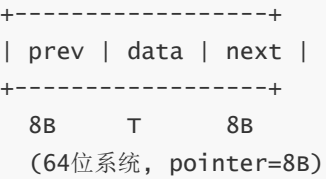
2.1 底层数据结构

list是带头结点的双向循环链表。

```
+--->[header]<---+
|      /      \      |
|      v        v      |
| [node1]  [node2] |
```



单个节点的内存布局：



遍历方式：

header->next->next->...->header
header->prev->prev->...->header

GCC中list的实现：

```
// GCC libstdc++源码简化：节点结构
struct _List_node_base {
    _List_node_base* _M_next;
    _List_node_base* _M_prev;
};

template<typename _Tp>
struct _List_node : public _List_node_base {
    _Tp _M_data;
};

// list类持有header节点
template<typename _Tp>
class list {
    _List_node_base _M_node;    // 哨兵/header节点
    // _M_node._M_next指向第一个元素
    // _M_node._M_prev指向最后一个元素
    // 空list时二者都指向_M_node自身
};
```

2.2 时间复杂度

操作	时间复杂度	说明
<code>push_front</code>	$O(1)$	头部插入，仅修改指针
<code>push_back</code>	$O(1)$	尾部插入，仅修改指针
<code>pop_front</code>	$O(1)$	头部删除
<code>pop_back</code>	$O(1)$	尾部删除
<code>insert</code> (任意位置)	$O(1)$	已知位置时只需修改指针
<code>erase</code> (任意位置)	$O(1)$	已知位置时只需修改指针

操作	时间复杂度	说明
随机访问 <code>l[i]</code>	不支持	没有operator[], 需用advance() O(n)
查找	O(n)	必须线性遍历
<code>splice</code>	O(1)	拼接两个list, 仅修改指针
<code>sort</code>	O(n log n)	使用归并排序
<code>size()</code>	O(1)	C++11起要求O(1)

2.3 空间复杂度

- 每个节点额外开销: 2个指针 (prev + next) = 16B (64位系统)
- 没有capacity概念, size()个节点恰好占用size()个节点的空间
- 空间开销公式: `size() * (sizeof(T) + sizeof(void*) * 2)`

2.4 内存特征

链表节点的内存分布 (不连续):
堆:
0x1000: [prev=null | data=A | next=0x2400]
0x2400: [prev=0x1000 | data=B | next=0x3800]
0x3800: [prev=0x2400 | data=C | next=null]

vs vector的连续内存:
0x1000: [A][B][C]

- 节点分散分配, 每个节点独立new/delete
- 缓存不友好: 遍历时CPU cache命中率低
- 内存碎片化: 多次new可能产生内存碎片

2.5 设计理由: 为什么不只用vector?

方面	vector	list
中间插入	O(n) 移动元素	O(1) 修改指针
迭代器失效	扩容/插入/删除后可能失效	只有被删除节点的迭代器失效
内存连续性	连续, CPU缓存友好	不连续, 缓存不友好
随机访问	O(1)	O(n)
splice/merge	O(n) 需拷贝	O(1) 仅改指针

设计哲学: 当场景是"频繁在中间插入/删除", 用list; 当需要"频繁随机访问", 用vector。

2.6 源码思想

```
// list的sort算法特色：使用归并排序而非快速排序
// 原因：
// 1. 链表无法O(1)随机访问，快排的partition效率低
// 2. 归并排序天然适合链表（不需要额外空间，原地修改指针即可）
// 3. 归并排序稳定，该稳定性在链表中几乎无额外开销

// 简化版list::sort逻辑（非递归，自底向上归并）：
template<typename T>
void list<T>::sort() {
    // 使用carry数组，每个元素存储一个已排序的、大小为2^i的子链表
    // 通过不断合并两个等长子链表实现归并
    // 实际上是对"磁带归并"的模拟
    list carry;
    list counter[64]; // 最多支持2^64-1个元素
    int fill = 0;

    while (!empty()) {
        carry.splice(carry.begin(), *this, begin());
        int i = 0;
        while (i < fill && !counter[i].empty()) {
            counter[i].merge(carry); // 合并等长子链表
            carry.swap(counter[i++]);
        }
        carry.swap(counter[i]);
        if (i == fill) ++fill;
    }
    for (int i = 1; i < fill; ++i)
        counter[i].merge(counter[i-1]);
    swap(counter[fill-1]);
}
```

为什么list::sort使用非递归归并排序？避免递归调用栈溢出，对于超大的list也能正常工作。

2.7 实际应用

场景	说明
LRU缓存	每次访问将元素移到头部，需要O(1)插入/删除
消息队列	生产者尾部插入，消费者头部取出 -> O(1)
撤销/重做	操作记录，支持中间任意位置删除
splice场景	多个链表的拼接和拆分操作
内存碎片整理	利用splice重新组织内存

2.8 面试题

Q1: list的迭代器失效情况?

A: 插入操作不会使任何迭代器失效。删除操作仅使被删除元素的迭代器失效。这是list相较于vector的重大优势。

Q2: 为什么list没有operator[]?

A: list底层是链表，不支持O(1)随机访问。如果提供operator[]，调用者可能误以为是O(1)操作而写出O(n^2)的代码。标准委员会刻意不提供，迫使程序员使用更明显的advance()或显式循环。

Q3: list::size()的时间复杂度?

A: C++11之前GCC中是O(n)（因为splice是O(1)必须用O(n)的size来补偿），C++11强制要求splice也是O(1)并且size()也是O(1)（标准要求）。

Q4: list和forward_list的区别?

A: list是双向链表，forward_list是单向链表。forward_list更省内存（每节点仅一个指针），但不能反向遍历，也没有push_back（但有insert_after）。

Q5: list::sort和std::sort的区别?

A: std::sort需要随机访问迭代器，list只有双向迭代器，所以不能用std::sort。list::sort用归并排序，std::sort用快排（或内省排序Introsort）。

Q6: C++中如何从list中间删除元素而不遍历整个链表?

A: 保存要删除元素的迭代器，然后erase该迭代器——O(1)。这就是list的核心价值：已知位置的插入删除是O(1)。

Q7: list的最大缺点是什么?

A: (1) 缓存不友好——遍历时CPU cache miss率高，实践上比vector慢很多；(2) 每个节点有额外16B指针开销；(3) 不能随机访问。

Q8: splice有什么用?

A: 将一个list中的部分元素移动到另一个list的指定位置，O(1)操作。例如将最近访问的元素移到头部（实现LRU）。

Q9: list如何实现merge?

A: 利用双链表的特点，merge是"破坏性"合并（源list被清空）。从两个已排序链表头部开始比较，将较小者splice到目标链表。

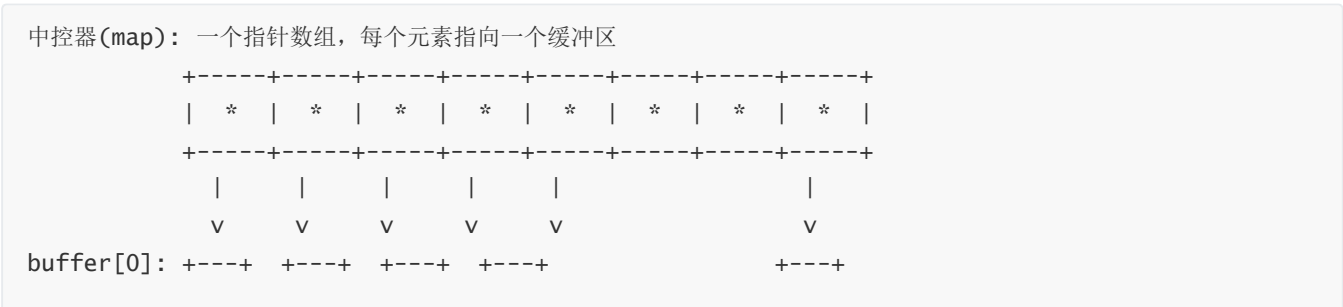
Q10: iterator和const_iterator的区别?

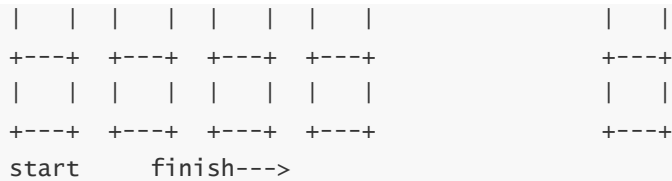
A: const_iterator不允许修改指向的元素（只读），iterator允许读写。list::iterator修改 `*it = value` 是合法的，而const_iterator不允许。const_iterator可以通过cbegin()/cend()获取。

3. deque

3.1 底层数据结构

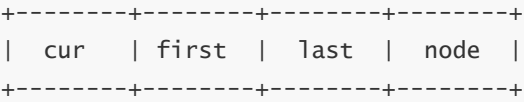
deque（double-ended queue）采用**分段连续空间（chunked array / map of pointers）**的设计。





每个缓冲区(buffer)大小固定（通常512B / sizeof(T) 或 8个元素）

迭代器结构：



cur : 指向当前元素
first: 指向当前缓冲区起始
last : 指向当前缓冲区末尾+1
node : 指向中控器中当前buffer的指针

3.2 时间复杂度

操作	时间复杂度	说明
<code>push_front</code>	$O(1)$	头部插入，可能触发map重分配
<code>push_back</code>	$O(1)$	尾部插入，可能触发map重分配
<code>pop_front</code>	$O(1)$	头部删除
<code>pop_back</code>	$O(1)$	尾部删除
随机访问 <code>d[i]</code>	$O(1)$	通过两步计算：先算buffer再算偏移
<code>insert</code> (中间)	$O(n)$	需要移动元素
<code>erase</code> (中间)	$O(n)$	需要移动元素
<code>insert</code> (头部/尾部)	$O(1)$	少数情况 $O(n)$ ——当需要扩容中控器

3.3 空间复杂度

- 中控器：一个指针数组，大小随元素增加而扩展
- 每个buffer：大小固定，`sizeof(T) * buffer_size`
- 尾部buffer和头部buffer通常有不完全利用的情况
- 空间利用率介于vector和list之间

3.4 扩容机制

deque的扩容分两层：

1. **buffer层面**：不需要扩容，直接在中控器前端或后端分配新的buffer
2. **中控器(map)层面**：当中控器指针数组满时，分配更大的指针数组，把旧的指针拷贝到中间

需求	vector	list	deque
随机访问	O(1)	O(n)	O(1) (略慢于vector)
头部插入	O(n)	O(1)	O(1)
尾部插入	O(1)	O(1)	O(1)
内存连续性	完全连续	不连续	分段连续
缓存友好	最好	最差	中等

需求	vector	list	deque
迭代器复杂度	指针	简单	复杂（4个指针）

3.7 源码思想

```

// deque迭代器的核心操作（GCC源码简化）
// 为什么O(1)随机访问稍慢于vector？
// 因为需要两步计算：

template<class T>
typename deque<T>::reference
deque<T>::operator[](size_type __n) {
    // 第一步：计算在第几个buffer
    // 第二步：计算在buffer内的偏移
    // 两步计算增加了一些开销
    return *(_M_impl._M_start + difference_type(__n));
}

// 迭代器 + n 的计算：
// self + n = self + n
// 如果 n > 0：向前跨buffer
// 跨buffer次数 = (n + 当前buf内剩余元素) / buffer_size
// 然后在最终buffer中定位到 cur + remaining_offset

```

3.8 实际应用

场景	说明
任务调度器	既有头部取任务又有尾部加任务
滑动窗口	需要在头尾频繁添加/删除
undo/redo	支持两端操作的历史记录
双端BFS	双向广度优先搜索需要双端操作

3.9 面试题

Q1: deque的底层数据结构是什么？

A: 分段连续空间（chunked array）。使用中控器（map，一个指针数组）管理多个固定大小的缓冲区（buffer），每个buffer是连续内存块。迭代器包含4个指针来描述当前位置。

Q2: deque如何实现O(1)随机访问？

A: `d[i]` 的计算分两步：(1) 确定目标元素在第几个buffer: `(i + front_offset) / buffer_size`，(2) 确定在buffer内的偏移: `(i + front_offset) % buffer_size`。虽然是O(1)，但比vector多一次间接寻址。

Q3: deque在什么情况下比vector慢？

A: (1) 随机访问：deque有额外的间接计算，(2) 遍历：buffer边界检查有开销，(3) 缓存：deque不是完全连续内存，缓存命中率略低。

Q4: deque适合做栈的底层容器吗?

A: 适合。deque是C++ stack的默认底层容器。比vector好因为不会在扩容时拷贝所有元素。而且deque不需要reserve()之类的管理。

Q5: deque有capacity()吗?

A: 没有。deque的内存是分段管理的，不像vector那样是一个大块连续内存。所以deque没有capacity的概念，也不需要reserve()。这也是deque相对于vector的一个设计优势。

Q6: deque的迭代器失效情况?

A: push_front/push_back不会使引用失效（但可能使迭代器失效）。insert/erase在中间会使所有迭代器失效。在头部/尾部插入时，引用不失效（因为元素在buffer中的位置不变），但迭代器可能失效（因为map可能扩容）。

Q7: 如何选择deque和vector?

A: 需要push_front就用deque；只需要push_back且需要完全连续内存用vector；数据量大且频繁扩容用deque（扩容开销小）；需要传给C接口（需要&data[0]）用vector。

Q8: deque的buffer大小如何确定?

A: GCC默认为：如果 sizeof(T) < 512，buffer_size = 512 / sizeof(T)；否则buffer_size = 1。这样保证每个buffer约512字节，兼顾缓存和灵活性。

Q9: deque和vector的区别有哪些?

A: deque支持push_front，vector不支持；deque没有capacity()和reserve()，vector有；deque元素非完全连续，vector完全连续；deque扩容代价更小（只需分配新buffer），vector需全局重新分配。

Q10: 能否用deque的数据传给C API（要求连续内存）？

A: 不能。deque的元素不是完全连续的。需要先拷贝到vector再传给C API。或者直接使用vector。

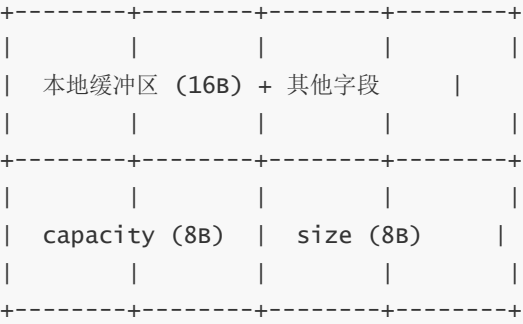
4. string

4.1 底层数据结构

string的底层是一个**支持SSO（Small String Optimization）的动态字符数组**。

SSO下的内存布局（GCC libstdc++，64位系统）：

sizeof(string) = 32 bytes



短字符串模式（长度 <= 15）：
[hello world\0.....] 本地buf直接存储
[cap=15] capacity=15
[size=11] size=11

```
长字符串模式 (长度 > 15):
[ptr to heap data.....] 指向堆上字符串
[cap=256                    ] capacity=256
[size=200                   ] size=200
```

4.2 SSO详解

GCC libstdc++ (g++): SSO阈值 = 15字符

```
// GCC std::string内部结构 (简化)
// sizeof(string) = 32 bytes
// _M_local_buf: 16 bytes
// _M_allocated_capacity: 8 bytes (size_t)
// _M_string_length: 8 bytes (size_t)

// 判断是否使用SSO: pointer是否指向_M_local_buf内部
bool _M_is_local() const {
    return _M_data() == _M_local_data();
}

// 本地缓冲区 (15个char + '\0' = 16字节)
// 最后一个字节是 '\0' 或用于区分长短模式
```

LLVM libc++ (clang++): SSO阈值 = 22字符

LLVM libc++的string设计更激进:

```
// LLVM libc++ std::string内部结构
// sizeof(string) = 24 bytes (64位系统)
// 结构更紧凑:
// - 长字符串模式: [ptr(8) | size(8) | cap(7) | is_long(1)]
// - 短字符串模式: [buf[0..22] | size(1)]

// 长字符串标志位占用cap的最高位
// 短字符串的size存在最后一个字节(高4位存放size)
// 本地缓冲区可以放22个字符(含'\0')
```

LLVM string内存布局(24字节):

长字符串模式:

```
+-----+-----+-----+
| ptr (8 bytes) | size (8 bytes) | cap(7)+flag(1) |
+-----+-----+-----+
```

短字符串模式:

```
+-----+-----+
|          字符缓冲区 (22 bytes)          |size(1)|
+-----+-----+
| h | e | l | l | o | \0 | ...          | 5 |
+-----+-----+
```

为什么是15/22字符？

实现	sizeof(string)	SSO阈值	推算
GCC	32B	15字符	32B - 8B(size) - 8B(cap) = 16B本地buf, 1B给'\0', 剩15B
Clang	24B	22字符	更激进的layout设计，复用字段
MSVC	32B	15字符	类似GCC但略有差异

为什么需要SSO？
大量字符串是短字符串（如key名字、URL path、JSON key等），SSO避免了为这些短字符串进行堆分配，显著提升性能。是典型的"小数据优化"。

4.3 时间复杂度

操作	时间复杂度	说明
随机访问 <code>s[i]</code>	$O(1)$	类似vector，连续内存
<code>push_back</code> (未扩容)	$O(1)$	尾部追加
<code>push_back</code> (扩容时)	$O(n)$	均摊 $O(1)$
<code>insert</code> (中间)	$O(n)$	需要移动后续字符
<code>erase</code> (中间)	$O(n)$	需要移动后续字符
<code>find</code>	$O(n*m)$	朴素字符串匹配；不能保证是最优算法
<code>substr</code>	$O(n)$	需要拷贝（C++11起， $O(k)$ 拷贝k个字符）
<code>c_str()</code> / <code>data()</code>	$O(1)$	返回内部指针
<code>+</code> / <code>+=</code>	$O(n+m)$	拼接操作

4.4 扩容机制

string的扩容和vector类似，但略有不同：

- GCC：通常是2倍扩容
- MSVC：通常是1.5倍扩容
- 不同的是：string扩容时还要维护C-style null终止符（\0）


```
// string扩容时要注意null-terminated
// 实际上capacity始终保证s[capacity()] = '\0'
// 但size()不包含'\0'

// 典型扩容代码：
size_type _M_check_length(size_type __n1, size_type __n2, const char* __s) const {
    if (max_size() - (size() - __n1) < __n2)
        __throw_length_error(__s);
    const size_type __len = size() + std::max(size(), __n2); // 2倍扩
    return std::min(__len, max_size());
}
```

4.5 内存布局对比

短字符串 (SSO)：

栈/堆上string对象：

内部buffer: [H][e][l][l][o][\0][...15个字节...]

size=5, cap=15

长字符串：

栈/堆上string对象：

ptr --> 堆: [H][e][l][l][o][][w][o][r][l][d][\0][...未用空间...]

size=11, cap=256

对比C风格字符串：

char* ptr --> 堆: [H][e][l][l][o][\0]

需要手动管理内存

4.6 设计理由

设计决策	理由
SSO	短字符串避免堆分配，显著提升性能
连续内存+c_str()	兼容C API (fopen, printf等)
结尾保证\0	兼容C字符串，c_str()是O(1)
引用计数COW被废弃	多线程环境下性能差，C++11明确禁止
move语义支持	避免深拷贝，O(1)转移所有权

关于COW (Copy-On-Write, 写时复制)：

早期的GCC std::string使用COW (引用计数共享底层数据，修改时才拷贝)。

但COW在多线程环境下引入了严重的性能问题 (每次读都需要原子操作操作引用计数)，而且实践中发现大多数场景下COW弊大于利。C++11起标准明确禁止COW实现。

4.7 源码思想

```
// string重要的设计技巧：利用traits类型进行编译期决策

// char_traits定义了字符的操作：
// - 比较、查找、拷贝、赋值、移动
// - 不同特化优化不同字符类型

// 如 char_traits<char>::length 直接调用strlen
// 但 char_traits<wchar_t>::length 调用wcslen

// 还有 std::char_traits<char>::copy vs std::char_traits<char>::move
// 分别调用memcpy和memmove，正确处理重叠
```

4.8 实际应用

场景	说明
文本处理	文件读写、字符串拼接
网络协议	HTTP报文构造和解析
JSON/XML	结构化数据的序列化/反序列化
日志系统	格式化字符串拼接
GUI编程	窗口标题、标签文本等

4.9 面试题

Q1: std::string能否存储'\0'字符？

A: 可以。std::string不是以'\0'判断结束的——它用size()成员维护长度。可以在任意位置包含'\0'字符。但c_str()返回的C风格字符串在遇到第一个'\0'后停止。

Q2: c_str()和data()的区别？

A: C++11之前，data()不保证以'\0'结尾（但实际实现都加了）。C++11起两者等价，都返回null-terminated的字符数组。C++17引入non-const的data()，允许直接修改内部缓冲区。

```
string s = "hello";
char* p = s.data(); // C++17: okay, 可修改
p[0] = 'H';         // 可以修改内部缓冲区
```

Q3: SSO是什么？GCC的string能存储多少字符不用堆分配？

A: SSO (Small String Optimization)。GCC 32B的string对象中，本地缓冲区16B，除去1B的'\0'，可存15个字符。LLVM libc++可存22个字符。MSVC可存15个字符。

Q4: 如何触发SSO和长字符串模式的切换？

A: 当字符串长度超过SSO阈值（如GCC的15）时，自动切换到堆模式。push_back或+=操作触发扩容时，检测到本地buf不够用，分配堆内存，把本地buf内容拷贝过去并释放本地buf。

Q5: C++11起为什么废弃COW?

A: (1) 多线程下原子操作引用计数开销大; (2) `operator[]`返回non-const引用, 无法判断是否要COW; (3) 实践中大多数操作都会触发copy, COW优势不大; (4) 移动语义提供了更高效的替代方案。

Q6: 如何高效拼接大量字符串?

A: 使用`ostringstream`, 或预先`reserve`容量后逐个`+=`, 或在C++20中用`fmt::format`。避免使用`+`拼接大量短字符串 (每次`+`都创建临时对象)。

```
// 不好: 每次+创建临时string
string result = s1 + s2 + s3 + s4; // 至少2个临时对象
// 好: 预估大小后reserve
string result;
result.reserve(s1.size() + s2.size() + s3.size() + s4.size());
result += s1; result += s2; result += s3; result += s4;
```

Q7: `to_string`的性能陷阱?

A: `to_string`在内部使用`sprintf`或等价方法, 返回新string。频繁调用`to_string`会导致大量临时对象, 影响性能。考虑使用`to_chars` (C++17) 或第三方库。

Q8: `std::string`和`std::string_view`的区别?

A: `string_view`是C++17引入的非拥有 (non-owning) 字符序列视图。它不管理内存, 只持有指针和长度。适合作为函数参数, 避免不必要的string拷贝。

```
void process(std::string_view sv) { // 不拷贝字符串
    size_t pos = sv.find(':');
    // ...
}
process("hello world"); // 不创建string
process(s);             // 不拷贝s
process(s.substr(0, 5)); // 不创建新的substr string
```

Q9: `std::string`的移动语义如何工作的?

A: 移动构造/赋值只复制指针、size、capacity, 然后把源对象置于有效但未定义状态 (空字符串或只有SSO缓冲区)。比深拷贝快得多。

Q10: 如何高效地将int/double转为string?

A: C++17的 `std::to_chars` (不去分配locale, 不抛异常, 不分配内存); C++20的 `std::format`。简单场景用 `to_string` 即可。

5. map / multimap

5.1 底层数据结构

map底层是**红黑树 (Red-Black Tree)**, 一种自平衡二叉搜索树。

红黑树节点结构:

```
+-----+
| parent(指向父节点)          |
| left  (指向左子节点)        |
| right (指向右子节点)        |
```

```
| color (RED or BLACK)      |
| key   (const Key)         |
| value (mapped_type)       |
+-----+

```

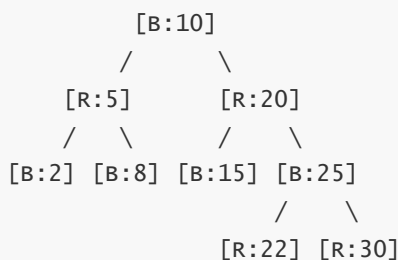
GCC源码中节点结构（简化）：

```
struct _Rb_tree_node_base {
    _Rb_tree_color _M_color;      // RED 或 BLACK
    _Rb_tree_node_base* _M_parent;
    _Rb_tree_node_base* _M_left;
    _Rb_tree_node_base* _M_right;
};

template<typename _Val>
struct _Rb_tree_node : _Rb_tree_node_base {
    _Val _M_value_field;
    // _Val = pair<const Key, T>
};

```

红黑树结构示意图：



红黑树五条性质：

1. 每个节点是红色或黑色
2. 根节点是黑色
3. 叶子节点(NIL)是黑色
4. 红色节点的两个子节点都是黑色（不能连续红色）
5. 从任一节点到其每个叶子节点的路径都包含相同数量的黑色节点

=> 这保证了：最长路径 $\leq 2 * \text{最短路径}$

=> 树的高度 $\leq 2 * \log(n+1)$ ，保证了 $O(\log n)$ 操作

5.2 时间复杂度

操作	红黑树时间	说明
插入	$O(\log n)$	最多2次旋转
删除	$O(\log n)$	最多3次旋转
查找	$O(\log n)$	递归搜索最多 $\log n$ 层
<code>begin()</code>	$O(\log n)$	找最左节点
<code>lower_bound</code>	$O(\log n)$	二分搜索

操作	红黑树时间	说明
<code>upper_bound</code>	$O(\log n)$	二分搜索
遍历所有元素	$O(n)$	中序遍历
<code>operator[]</code>	$O(\log n)$	包含查找+插入两个操作
<code>equal_range</code>	$O(\log n)$	搜索边界

5.3 空间复杂度

- 每个节点开销：parent指针(8B) + left指针(8B) + right指针(8B) + color(1B但内存对齐后4或8B) = 约32B/节点
- 总空间：`n * (sizeof(Key) + sizeof(Value) + 32B)`

5.4 红黑树 vs AVL树

属性	红黑树	AVL树
平衡条件	宽松（最长路径 $\leq 2 \times$ 最短路径）	严格（
查找性能	$O(\log n)$ ，略慢于AVL	$O(\log n)$ ，略快于红黑树
插入旋转	最多2次旋转	可能需要多次旋转（回溯到根）
删除旋转	最多3次旋转	可能需要 $O(\log n)$ 次旋转
适用场景	插入/删除频繁；C++ STL	查找频繁
实现复杂度	复杂	复杂

为什么STL选择红黑树？

- STL容器通常插入和查找频率相当，红黑树在两者之间取得最佳平衡
- AVL树虽然查找更快，但插入/删除的旋转成本高
- 实测显示，综合性能红黑树优于AVL

5.5 设计理由

为什么不用二叉搜索树（BST）？

单纯BST可能退化为链表（如顺序插入1,2,3,4,5...），查找从 $O(\log n)$ 退化为 $O(n)$ 。

为什么不用有序数组？

有序数组查找 $O(\log n)$ ，但插入 $O(n)$ （所有元素后移），不适合频繁插入场景。

为什么不用哈希表？

哈希表查找 $O(1)$ 平均，但不支持有序遍历。map的设计目标之一就是按key顺序遍历。

5.6 源码思想

```
// map的核心技巧：利用红黑树的lower_bound实现operator[]
// 如果key不存在，则插入默认值并返回

template<typename _Key, typename _Tp>
_Tp& map<_Key, _Tp>::operator[](const _Key& __k) {
    // 1. lower_bound找到插入位置
    iterator __i = lower_bound(__k);
    // 2. 如果key存在，直接返回value的引用
    // 3. 如果key不存在，在该位置插入pair(key, T())
    if (__i == end() || key_comp()(__k, (*__i).first))
        __i = insert(__i, value_type(__k, mapped_type()));
    return (*__i).second;
}

// 为什么key是const?
// pair<const Key, T> - 修改key会破坏红黑树的有序性质
// 这就是为什么不能用iterator修改key
```

红黑树插入后调整（简化解）：

插入红节点后需要修复的情况：

Case 1：叔叔节点是红色 → 改变父和叔为黑，祖父为红，向上继续修复

Case 2：叔叔节点是黑色，且新节点在父节点的"内侧" → 先旋转使之外侧

Case 3：叔叔节点是黑色，且新节点在父节点的"外侧" → 旋转并调整颜色，结束

每种情况都保证：不破坏性质5（黑高不变），逐步消除性质4违规（连续红色）

5.7 实际应用

场景	说明
字典/配置	按key有序存储配置项
区间查询	lower_bound/upper_bound查找范围
带顺序的缓存	按访问时间排序
符号表	编译器中的符号查找
有序统计	使用lower_bound实现排名查询

5.8 面试题

Q1: map的key为什么是const?

A: 因为map基于红黑树实现，树的排序依赖key的比较。如果允许修改key，会破坏红黑树的排序性质，导致未定义行为。如果需要修改key，可以先erase再insert。

Q2: map和unordered_map如何选择?

A: 需要有序遍历选map; 仅需要O(1)查找不关心顺序选unordered_map; 数据量大且哈希函数好选unordered_map (查找快); 数据量小选map (常数因子小, 无哈希计算开销); key不能哈希 (如pair<int, int>) 选map (只需提供operator<)。

Q3: map::operator[]和insert的区别?

A:

- `m[key]`: 如果key存在返回引用, 不存在则插入 `pair(key, T())` 并返回引用。总是返回引用, 且会覆盖已存在的值 (赋值语义)。
- `m.insert({key, val})`: 返回 `pair<iterator, bool>`。如果key已存在则插入失败, 保留原值。不覆盖。

Q4: map的迭代器失效情况?

A: 插入操作不使任何迭代器失效。删除操作仅使被删除元素的迭代器失效。不使其他任何迭代器失效。这是红黑树的优势。

Q5: map的lower_bound和upper_bound的复杂度?

A: 都是O(log n)。红黑树本身就是有序的, 从根出发, 根据key大小决定向左还是向右, 找到第一个 $\geq$ key (lower_bound) 或第一个 $>$ key (upper_bound) 的位置。

Q6: 为什么红黑树的插入/删除复杂度是O(log n)?

A: 红黑树的高度最多是 $2 \cdot \log(n+1)$ (因为最长路径不超过最短路径的2倍), 所以从根到叶最多走O(log n)层。旋转操作仅修改常数个指针, O(1)。

Q7: 红黑树和跳表(Skip List)有什么区别?

A: 跳表用层级随机化实现O(log n)查找, 实现更简单 (不需要旋转), Redis的SortedSet就用跳表。但跳表有随机性 (可能退化), 且空间开销更大 (每层额外指针)。

Q8: map能否自定义比较函数?

A: 可以。通过模板参数指定:

```
struct CaseInsensitiveCompare {
    bool operator()(const string& a, const string& b) const {
        return lexicographical_compare(
            a.begin(), a.end(), b.begin(), b.end(),
            [](char c1, char c2) { return tolower(c1) < tolower(c2); }
        );
    }
};

map<string, int, CaseInsensitiveCompare> m;
```

Q9: map和set的区别?

A: map存储pair<const Key, T> (key-value对), set仅存储key。底层都是红黑树。set没有operator[], 也不允许重复元素 (有multiset允许重复)。

Q10: multimap有什么特殊之处?

A: multimap允许重复key, 没有operator[] (因为有多多个值可能对应同一个key)。equal_range()返回一对迭代器, 表示所有同key元素的范围。

6. unordered_map

6.1 底层数据结构

unordered_map底层是**哈希表 (Hash Table)**，使用**开链法 (Separate Chaining)** 解决哈希冲突。

哈希表bucket结构：

buckets数组（指针数组）：

+-----+-----+-----+-----+-----+-----+-----+-----+							
bucket	bucket	bucket	bucket	bucket	bucket	bucket	bucket
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
+-----+-----+-----+-----+-----+-----+-----+-----+							
v	v		v				v
[k1,v1]	[k90,v90]		[k42,v42]				[k7,v7]
v			v				
[k9,v9]			[k58,v58]				
			v				
			[k74,v74]				

hash(key) % bucket_count = bucket_index

同一个bucket下的节点用单链表连接

GCC libstdc++ 哈希表实现结构：

_Hashtable 包含：

- _M_buckets: bucket数组的指针（指向指针数组）
- _M_bucket_count: bucket的数量
- _M_element_count: 当前元素个数
- _M_before_begin: 一个特殊节点，标记链表开始之前
- _M_single_bucket: 单bucket（最小容量时的优化）

节点结构（_Hash_node）：

+-----+	
_M_next	--> 指向下一个节点（单链表）
+-----+	
_M_storage	--> 存储实际的value(pair<const Key, T>)
+-----+	

GCC使用单链表存储所有节点（不仅仅是bucket内部）：

所有节点组成一个全局单链表：

`[_before_begin] -> [k1,v1] -> [k9,v9] -> [k90,v90] -> ...`

`buckets`数组只存储指向`bucket`内第一个节点的指针：

`bucket[0] -> [k1,v1]`

`bucket[1] -> [k90,v90]`

`bucket[2] -> [k42,v42]`

`...`

6.2 哈希冲突解决方法

方法	描述	优点	缺点	STL使用?
开链法(Separate Chaining)	每个bucket维护链表	简单，负载因子可>1	额外指针开销	Yes (GCC/Clang)
线性探测(Linear Probing)	冲突时找下一个空slot	缓存友好	集群现象 (clustering)	No
二次探测(Quadratic Probing)	冲突时跳跃探测	减少clustering	仍可能有 clustering	No
双重哈希(Double Hashing)	用第二个哈希函数	分布最均匀	计算开销大	No

6.3 时间复杂度

操作	平均	最差	说明
查找 <code>find</code>	$O(1)$	$O(n)$	哈希函数差时可能退化为 $O(n)$
插入 <code>insert</code>	$O(1)$	$O(n)$	负载因子过高触发rehash时 $O(n)$
删除 <code>erase</code>	$O(1)$	$O(n)$	平均 $O(1)$
<code>operator[]</code>	$O(1)$	$O(n)$	同find+可能的insert

6.4 空间复杂度

- bucket数组: `bucket_count * sizeof(pointer)`
- 每个节点: `sizeof(pointer) + sizeof(pair<const Key, T>)` (单链表为一个next指针)
- 总空间约: `bucket_count * 8B + n * (8B + sizeof(pair<const Key, T>))`

6.5 扩容机制 (Rehashing)

rehash过程：

- 当 `load_factor = element_count / bucket_count >= max_load_factor` 时触发
GCC中: `max_load_factor`默认 = 1.0

2. 分配新的bucket数组：
`new_bucket_count = next_prime(old_bucket_count * 2)`
 （通常取下一个质数）
3. 对所有元素重新哈希：
`for each element:`
`new_hash = hash(element.key) % new_bucket_count`
 将元素插入 `new_buckets[new_hash]`
4. 释放旧bucket数组

负载因子阈值设定理由：

- 太低（如0.5）：浪费空间，但冲突少，查找很快
- 太高（如2.0）：节省空间，但冲突多，查找可能O(n)
- 默认1.0：工程上的平衡点

6.6 设计理由：为什么不是map也不是vector？

需求	unordered_map	map	vector(有序)
查找	O(1)均摊	O(log n)	O(log n)
有序	否	是	是
内存连续性	差	差	好
迭代器稳定性	rehash后失效	插入不失效	很多情况失效
适用	纯查找	有序+查找	极少修改

6.7 源码思想

```
// GCC libstdc++ unordered_map实现要点（简化）

// 1. 质数的选择
// GCC内部有一个预计算的质数表
static const unsigned long __prime_list[] = {
    2u, 3u, 5u, 7u, 11u, 13u, 17u, 19u, 23u, 29u, 31u,
    // ... 一直到大质数
    4294967291u // 接近2^32的最大质数
};

// 2. 为什么bucket数量用质数？
// 减少哈希冲突——如果bucket数是2的幂，hash%2^k只依赖低位bit
// 如果哈希函数低位分布不均，冲突率会很高
// 质数取模会让所有bit都参与结果计算

// 3. insert的详细流程（简化版）：
pair<iterator, bool> _M_insert_unique(const value_type& __v) {
    // 检查是否需要rehash
    if (size() > _M_bucket_count * _M_max_load_factor) {
        rehash(next_prime(_M_bucket_count + 1));
    }
}
```

```

    }
    // 计算hash -> 找到bucket -> 检查是否已存在
    // 如果不存在 -> 在bucket头部插入新节点
}

// 4. 单链表插入优化：总是在bucket头部插入（O(1)）
// 不需要遍历到链表尾部

```

6.8 实际应用

场景	说明
key-value缓存	O(1)查找，不需要有序
去重和计数	MapReduce中的word count
数据库索引	哈希索引，等值查询
配置系统	根据key快速查找配置项
图算法	邻接表用unordered_map实现

6.9 面试题

Q1: unordered_map如何解决哈希冲突？

A: 使用开链法（Separate Chaining）。每个bucket维护一个单链表，冲突的元素挂在同一个bucket的链表上。查找时先定位bucket再遍历链表。

Q2: 负载因子(load_factor)是什么？如何影响性能？

A: $\text{load_factor} = \text{size}() / \text{bucket_count}()$ 。GCC默认 $\text{max_load_factor} = 1.0$ 。当超过阈值时触发rehash。负载因子过高导致链表变长、查找退化；过低则浪费空间。

Q3: rehash过程中会发生什么？

A: (1) 分配新的bucket数组(bucket_count通常是旧数组的两倍的下一个质数)；(2) 将旧数组所有元素重新计算hash值按新bucket_count取模放入新bucket；(3) 释放旧bucket数组。rehash会使所有迭代器失效。

Q4: unordered_map::reserve()的作用？

A: 预先分配足够的bucket以保证可以容纳n个元素而不触发rehash。与vector::reserve()类似，避免多次rehash的性能浪费。

Q5: 为什么自定义类型作为key需要提供hash函数和operator==？

A: hash函数决定元素放入哪个bucket；operator==用于在bucket链表中匹配具体元素（处理哈希冲突——同一个bucket可能有多个key值相同或不同但哈希冲突的元素）。

Q6: unordered_map的迭代器失效情况？

A: insert可能使所有迭代器失效（如果触发rehash）。erase仅使被删除元素的迭代器失效。这与红黑树的map不同（map的insert不使任何迭代器失效）。

Q7: 和map相比，unordered_map什么时候更快？什么时候更慢？

A: 更快：元素数量大（1000+），哈希函数质量好时，O(1) vs O(log n)差距明显；更慢：元素数量小时，map的常数因子小（不需要计算哈希）；极端情况：哈希函数差导致大量冲突时退化为O(n)。

Q8: 如何为unordered_map设计高效的自定义哈希函数?

A: 考虑: (1) 均匀分布——尽量让所有bucket均匀装载; (2) 计算快速——不要做昂贵操作; (3) 避免碰撞——不同的key尽量映射到不同hash值。

```
struct MyHash {
    size_t operator()(const MyKey& k) const {
        // 组合hash: 使用黄金比例或位运算混合多个字段
        size_t h1 = hash<int>()(k.field1);
        size_t h2 = hash<string>()(k.field2);
        return h1 ^ (h2 << 1); // XOR组合
    }
};
```

Q9: bucket_count为什么要质数?

A: 假设bucket_count是2的幂 (如256), `hash % 256` 只使用hash值的低8位, 如果hash值的低位分布不均, 冲突率很高。质数取模会用hash值的所有位参与计算, 分布更均匀。

Q10: 为什么GCC用单链表而不是双链表?

A: 节省空间 (每节点少一个prev指针); 插入总是在bucket头部 (不需要删除尾部); 可以维护全局单链表实现迭代器的递增遍历。

7. set / multiset

7.1 底层数据结构

set的底层与map完全相同: **红黑树**。区别是set的value就是key本身。

```
// GCC中set就是对红黑树的薄包装
// set和map共用同一套_Rb_tree基础设施

template<typename _Key, typename _Compare, typename _Alloc>
class set {
    // 内部就是一个红黑树, value_type = key_type
    _Rb_tree<key_type, value_type, _Identity<value_type>,
            key_compare, _Alloc> _M_t;
};

template<typename _Key, typename _Tp, typename _Compare, typename _Alloc>
class map {
    // 内部红黑树, value_type = pair<const Key, T>
    _Rb_tree<key_type, value_type, _Select1st<value_type>,
            key_compare, _Alloc> _M_t;
};

// _Identity: 直接返回自身 (用于set)
// _Select1st: 返回pair.first (用于map)
```

7.2 时间复杂度

操作	时间复杂度	说明
<code>insert</code>	$O(\log n)$	红黑树节点插入
<code>erase</code>	$O(\log n)$	红黑树节点删除
<code>find</code>	$O(\log n)$	树搜索
<code>count</code>	$O(\log n)$	等同于find
<code>lower_bound</code>	$O(\log n)$	二分查找边界
<code>upper_bound</code>	$O(\log n)$	二分查找上界
遍历	$O(n)$	中序遍历自然有序

7.3 空间复杂度

与map类似，每个节点约32B额外开销（颜色+3个指针），但value就是key，没有额外的mapped_value。

7.4 设计理由

set本质是"只有key没有value的map"。为什么不直接用map<Key, bool>或map<Key, void>？因为set的 `value_type == key_type`，遍历和算法使用更自然。

```
set<int> s = {3, 1, 4, 1, 5, 9};
// s = {1, 3, 4, 5, 9}  -- 自动排序+去重

map<int, bool> m;
m[3] = true; m[1] = true; m[4] = true; m[1] = true;
// 浪费了bool的存储
```

7.5 实际应用

场景	说明
去重	自动去除重复元素
有序集合	需要排序且无重复
区间查询	<code>lower_bound/upper_bound</code>
交集/并集/差集	<code>set_intersection</code> 等算法
白名单/黑名单	快速判断元素是否在集合中

7.6 面试题

Q1: set和map的底层区别？

A: 底层完全一样，都是红黑树。set的value_type=key_type；map的value_type=pair<const Key, T>。共用同一套_Rb_tree模板。

Q2: set为什么没有operator[]？

A: set只有key，没有mapped_value。operator[]的语义是"通过key访问mapped_value"，对于只有key的set没有意义。

Q3: set和unordered_set怎么选？

A: 需要有序用set；只需要O(1)查找用unordered_set；数据量小时用set（常数因子小）；数据量大且哈希函数好用unordered_set。

Q4: multiset和set的区别？

A: multiset允许重复元素。insert永远成功（不返回bool）。equal_range可获取所有相同元素的迭代器范围。count可能大于1。

Q5: 如何用set实现去重并保持顺序？

A: set自动做这两件事。插入时O(log n)确保无重复且有排序。

Q6: set的insert返回值是什么？

A: pair<iterator, bool>。iterator指向已存在或新插入的元素；bool表示是否成功插入（true=成功，false=元素已存在所以未插入）。

Q7: 如何实现自定义类型的set？

A: 提供operator<或自定义比较函数。

```
struct Person {
    string name;
    int age;
    bool operator<(const Person& other) const {
        return name < other.name; // 按名字排序和去重
    }
};
set<Person> people;
```

Q8: 为什么set遍历是有序的？

A: 因为红黑树的中序遍历（左->根->右）自然按key升序输出。

Q9: 如何高效获取set中第k小的元素？

A: 标准set不支持O(log n)的按排名访问。需要自己实现OrderedSet（红黑树节点维护子树大小），或使用GCC扩展的pb_ds中的tree。

Q10: set如何判断元素存在？

A: s.find(key) != s.end() 或 s.count(key) > 0（multiset中count可能>1）。C++20引入s.contains(key)。

8. queue

8.1 底层数据结构

queue是容器适配器（Container Adapter），默认底层容器是deque。

queue 是对 deque/list 的接口限制：

deque支持：
push_back push_front pop_back pop_front

queue限制为（FIFO）：
push = push_back (从尾部入队)
pop = pop_front (从头部出队)
front = 取头部元素
back = 取尾部元素

stack限制为（LIFO）：
push = push_back (从尾部入栈)
pop = pop_back (从尾部出栈)
top = 取尾部元素

```
// GCC源码简化
template<typename _Tp, typename _Sequence = deque<_Tp>>
class queue {
protected:
    _Sequence c; // 底层容器
public:
    void push(const value_type& __x) { c.push_back(__x); }
    void pop() { c.pop_front(); }
    reference front() { return c.front(); }
    reference back() { return c.back(); }
    bool empty() const { return c.empty(); }
    size_type size() const { return c.size(); }
};
```

8.2 时间复杂度

操作	时间复杂度	说明
push	O(1)	容器尾部操作
pop	O(1)	容器头部操作
front	O(1)	取队头元素
back	O(1)	取队尾元素
empty	O(1)	委托给底层容器
size	O(1)	委托给底层容器

8.3 设计理由

为什么queue默认用deque而不是list或vector?

容器	作为FIFO队列的理由	不足
deque	push_back O(1), pop_front O(1)	完美匹配
list	push_back O(1), pop_front O(1)	每次pop都要delete节点, 开销大
vector	push_back O(1)摊销	pop_front O(n)! 不可接受

8.4 实际应用

场景	说明
BFS	广度优先搜索
消息队列	生产者消费者模型
打印队列	先进先出处理
缓存淘汰	FIFO淘汰策略
任务调度	公平调度 (先到先处理)

8.5 面试题

Q1: queue的默认底层容器是什么? 为什么?

A: deque。因为deque同时支持高效的push_back和pop_front (O(1))。vector的pop_front是O(n) (需要移动所有元素)，不适合做队列。

Q2: queue能否用list作为底层容器?

A: 可以。 `queue<int, list<int>> q;` 但list每次push都会new节点，比deque开销大。

Q3: queue有迭代器吗? 为什么?

A: 没有。queue是容器适配器，只暴露FIFO接口 (front/back/push/pop)，隐藏了底层容器的迭代器。这符合封装原则——队列语义下不应该对中间元素进行随机访问。

Q4: queue和deque的区别?

A: deque是完整的双端队列容器 (支持随机访问、任意位置插入删除)。queue是deque的接口子集，只允许尾部插入和头部删除。

Q5: 如何实现一个固定大小的循环队列?

A: 用vector+两个索引 (head和tail)，模运算实现循环。或者用boost::circular_buffer。

9. stack

9.1 底层数据结构

stack同样是容器适配器，默认底层容器也是deque。

9.2 时间复杂度

操作	时间复杂度	说明
push	O(1)	容器尾部操作
pop	O(1)	容器尾部操作
top	O(1)	取栈顶元素
empty	O(1)	委托给底层容器
size	O(1)	委托给底层容器

9.3 设计理由

为什么stack默认也用deque？

容器	作为栈的理由	不足
deque	push_back/pop_back O(1)	完美匹配
vector	push_back/pop_back O(1)	扩容时需拷贝，不如deque
list	push_back/pop_back O(1)	每个节点额外指针开销

实际上vector和deque做栈的性能非常接近。vector的优势是内存连续、缓存友好；deque的优势是扩容时不需要整体拷贝。实践中差距不大。

9.4 实际应用

场景	说明
函数调用栈	递归模拟
表达式求值	中缀转后缀、计算器
括号匹配	编译器语法检查
DFS	深度优先搜索
undo/redo	编辑器的撤销重做
浏览器的前进后退	页面访问栈

9.5 面试题

Q1: stack默认用deque，为什么不直接用vector？

A: deque做栈：push/pop都是O(1)，不需要整体扩容（只需分配新buffer）。vector做栈：虽然push/pop也是O(1)，但扩容时需要把整个数组拷贝走。不过实践中两者性能接近。

Q2: stack有迭代器吗？

A: 没有。stack是LIFO容器适配器，只暴露top/push/pop接口。不能遍历栈内元素。

Q3: 递归和栈的关系？

A: 函数的递归调用使用调用栈（call stack）。每次函数调用压栈（压入参数、返回地址、局部变量），返回时弹栈。递归层数过多会导致栈溢出（stack overflow）。可以用显式的stack容器将递归转迭代避免了栈溢出。

Q4: 如何用两个队列实现栈？

A:

- push: 将元素加入queue1
- pop: 将queue1前n-1个元素移到queue2，取出queue1最后一个元素，然后swap(queue1, queue2)

或等效地:

- push: 将元素加入queue2，将queue1的所有元素移到queue2，然后swap
- pop: 从queue1取

Q5: 栈的应用——括号匹配？

A: 遇到左括号压栈，遇到右括号弹栈并检查是否匹配。最后栈为空则匹配成功。

10. priority_queue

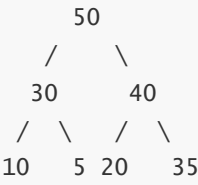
10.1 底层数据结构

priority_queue是容器适配器，底层默认使用vector实现二叉最大堆（Max Binary Heap）。

二叉堆的数组表示（用vector存储）：

```
索引：  0    1    2    3    4    5    6
      +---+---+---+---+---+---+
      | 50 | 30 | 40 | 10 | 5  | 20 | 35 |
      +---+---+---+---+---+---+
```

对应树结构：



性质：

- 对于节点i：
 - 父节点：(i-1)/2
 - 左子节点：2*i + 1
 - 右子节点：2*i + 2

- 最大堆：父节点 $\geq$ 所有子节点
- 根节点(索引0)是最大值

10.2 时间复杂度

操作	时间复杂度	说明
<code>push</code>	$O(\log n)$	插入然后上浮(sift-up/up-heap)
<code>pop</code>	$O(\log n)$	删除堆顶，最后一个元素移到顶然后下沉(sift-down/down-heap)
<code>top</code>	$O(1)$	直接返回索引0的元素
<code>empty</code>	$O(1)$	<code>vector::empty()</code>
<code>size</code>	$O(1)$	<code>vector::size()</code>

10.3 空间复杂度

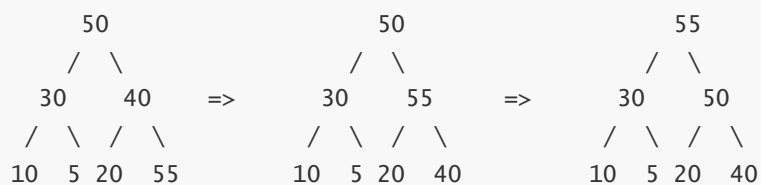
就是底层vector的空间。 $O(n)$ 。没有额外指针开销（堆用数组索引而不是指针维护父子关系）。

10.4 堆操作详解

push操作（上浮）：

1. 将新元素添加到数组末尾
2. 与父节点比较，如果大于父节点则交换
3. 重复步骤2直到 新元素 $\leq$ 父节点 或 到达根节点

示例：push(55)

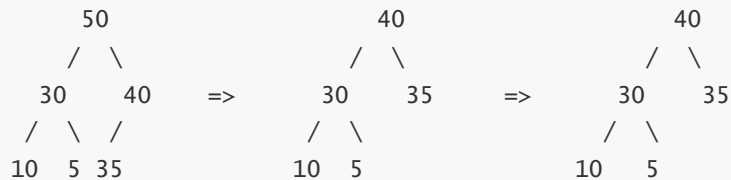


时间复杂度： $O(\log n)$ ，最多上浮 $\log n$ 层

pop操作（下沉）：

1. 取走堆顶元素（索引0）
2. 将最后一个元素（索引size-1）移到堆顶
3. 新堆顶与较大子节点比较，如果小于子节点则交换
4. 重复步骤3直到 新值 >= 子节点 或 到达叶子

示例：pop()



时间复杂度： $O(\log n)$ ，最多下沉 $\log n$ 层

10.5 设计理由

为什么不用有序数组/链表做优先队列？

实现方式	push	pop	top
二叉堆	$O(\log n)$	$O(\log n)$	$O(1)$
有序数组	$O(n)$ 需移动元素	$O(1)$ 取尾部	$O(1)$
排序链表	$O(n)$ 找插入位置	$O(1)$ 取头部	$O(1)$
二叉搜索树	$O(\log n) \sim O(n)$	$O(\log n) \sim O(n)$	$O(\log n)$

二叉堆在push和pop上都达到 $O(\log n)$ ，且实现简单（只用了vector，无指针开销），是最佳的平衡。

为什么不用AVL/红黑树？

AVL/红黑树的push和pop也是 $O(\log n)$ ，但每个节点有额外指针开销，且常数因子比二叉堆大。二叉堆仅用数组下标维护父子关系，零指针开销，缓存友好。

heap做成容器适配器而非独立容器的原因？

堆只需要操作top + push + pop三个操作，不涉及遍历/迭代器/查找。做成适配器符合“只暴露必须接口”的设计原则。

10.6 源码思想

```

// GCC libstdc++ make_heap核心算法（简化）

// 下沉操作：假设除了根节点外，左右子树都已是堆
// 调整根节点使整个树成为堆
template<typename __RandomAccessIterator>
void __adjust_heap(__RandomAccessIterator __first, __DifferenceType __holeIndex,
                  __DifferenceType __len, __Tp __value) {
    const __DifferenceType __topIndex = __holeIndex;
    __DifferenceType __secondChild = __holeIndex;
    while (__secondChild < (__len - 1) / 2) {
        __secondChild = 2 * (__secondChild + 1); // 右子节点
        // 选择较大的子节点
        if (*(__first + __secondChild) < *(__first + (__secondChild - 1)))
    
```

```

        __secondChild--;
        // 将较大的子节点上移
        *(__first + __holeIndex) = _GLIBCXX_MOVE(*(__first + __secondChild));
        __holeIndex = __secondChild;
    }
    // 处理边界情况（只有一个子节点）
    if ((__len & 1) == 0 && __secondChild == (__len - 2) / 2) {
        __secondChild = 2 * (__secondChild + 1);
        *(__first + __holeIndex) = _GLIBCXX_MOVE(*(__first + (__secondChild - 1)));
        __holeIndex = __secondChild - 1;
    }
    // 将value放到最终位置
    std::__push_heap(__first, __holeIndex, __topIndex, _GLIBCXX_MOVE(__value));
}

// 上浮操作：
template<typename _RandomAccessIterator>
void __push_heap(_RandomAccessIterator __first, _DifferenceType __holeIndex,
                _DifferenceType __topIndex, _Tp __value) {
    _DifferenceType __parent = (__holeIndex - 1) / 2;
    // 找到value应该放置的位置
    while (__holeIndex > __topIndex
        && *(__first + __parent) < __value) {
        *(__first + __holeIndex) = _GLIBCXX_MOVE(*(__first + __parent));
        __holeIndex = __parent;
        __parent = (__holeIndex - 1) / 2;
    }
    *(__first + __holeIndex) = _GLIBCXX_MOVE(__value);
}

```

make_heap的复杂度：线性时间 $O(n)$ 而非 $O(n \log n)$!

make_heap：从最后一个非叶子节点开始，逐个下沉
位置： $n/2 - 1$, $n/2 - 2$, ..., 0

虽然每个节点可能下沉 $O(\log n)$ ，但大部分节点离叶子近，下沉很少层
可以证明总操作次数 $\leq n$ （等比数列求和）

10.7 实际应用

场景	说明
任务调度	优先级最高的任务先执行
Dijkstra算法	每次取距离最小的节点
Huffman编码	构建Huffman树的合并过程
Top-K问题	最小堆维护最大的K个元素
事件模拟	按时间顺序处理事件

场景	说明
数据流中位数	用两个堆维护

10.8 面试题

Q1: priority_queue默认是比较大的优先，如何改为小的优先？

A: 使用greater比较器：

```
priority_queue<int, vector<int>, greater<int>> min_heap;
// 或者对自定义类型：
priority_queue<Task, vector<Task>, function<bool(Task, Task)>> pq(
    [](const Task& a, const Task& b) { return a.priority > b.priority; }
);
```

Q2: 二叉堆和二叉搜索树的区别？

A: 二叉堆只保证父>=子，不保证左<右。二叉搜索树保证左<根<右。二叉堆用数组存储（下标找子节点），搜索树用指针。二叉堆top() O(1)，搜索树最值也是O(log n)。

Q3: make_heap是线性时间的证明思想？

A: 自底向上做下沉。第h层有 2^h 个节点，每个节点下沉最多(log n - h)次。总次数 = $\sum(2^h * (\log n - h))$ ，求和结果 $\leq n$ 。直观理解：越高的层节点越少，下沉次数多；越低的层节点多，但下沉次数少。

Q4: 如何用堆实现Top-K问题（找最大的K个元素）？

A: 用最小堆维护K个元素。对于每个新元素：(1) 如果堆元素数<K则直接push；(2) 如果新元素 > 堆顶，pop堆顶再push新元素。最后堆中即为最大的K个元素。复杂度O(n log k)，远优于全排序的O(n log n)。

Q5: 如何用堆维护数据流的中位数？

A: 维护两个堆：最大堆存较小的一半数，最小堆存较大的一半数。两个堆大小之差不超过1。中位数 = 较大堆的堆顶（两堆大小不等时）或两个堆顶的平均值。

Q6: 为什么没有priority_queue的迭代器？

A: priority_queue的语义是优先级有序而非全序排列。底层vector中的元素并不是完全有序的（仅维持堆性质），迭代访问没有意义。仅暴露top()。

Q7: 堆排序 (heapsort) 什么时候用？

A: 需要O(n log n)排序但不想用额外空间时（原地排序）。但堆排序实践中通常比快排慢（缓存不友好，跳跃访问）。通常用Introsort（快排+堆排+插入排的混合）。

Q8: 为什么priority_queue用vector不是deque？

A: 堆操作需要频繁的O(1)随机访问（父子节点通过下标计算），vector在这点上和deque一样好。但vector内存完全连续，对缓存更友好。且deque的buffer边界检查在堆操作中无帮助反而有开销。

Q9: push_heap和pop_heap的算法复杂度？

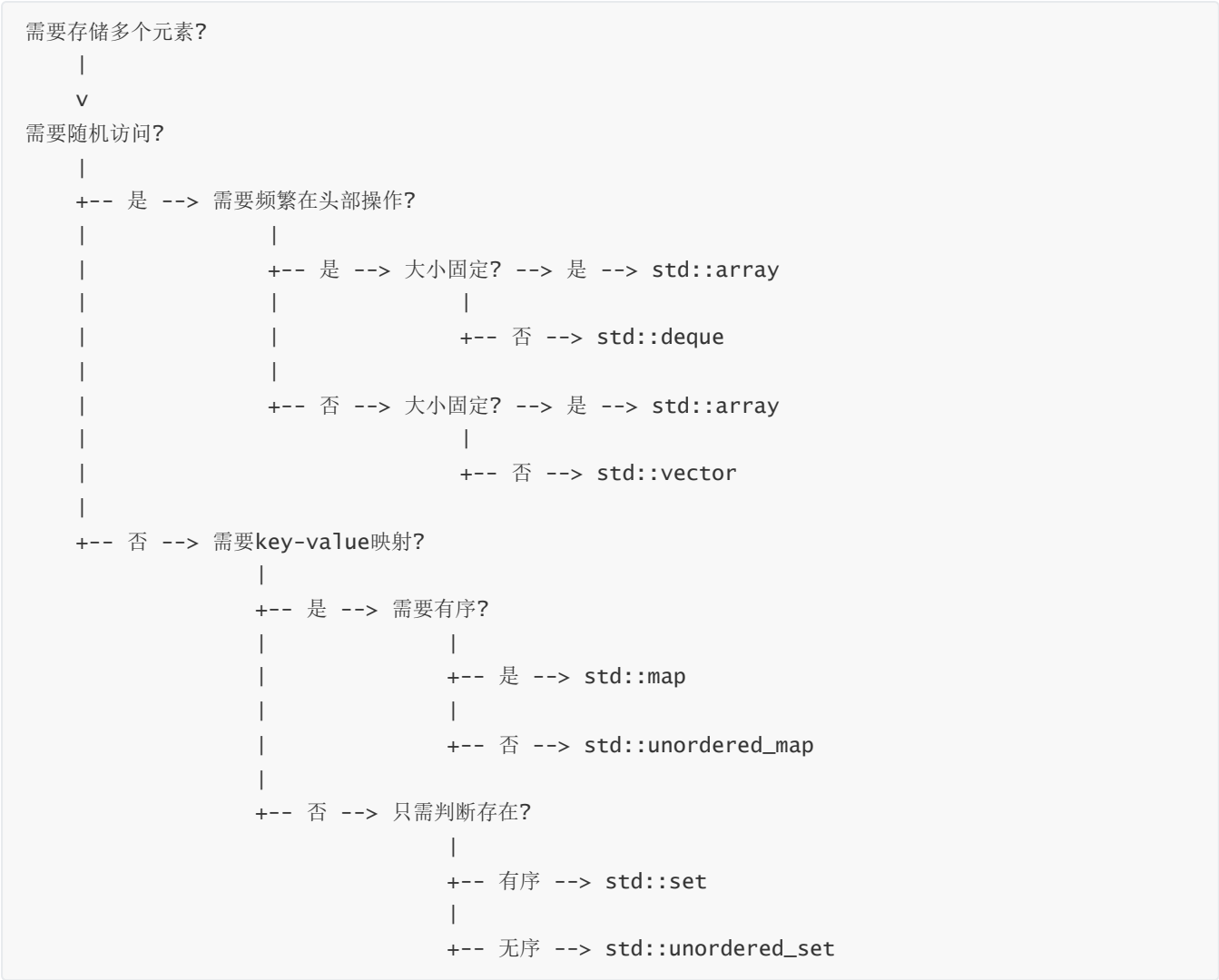
A: push_heap（上浮）：O(log n)，最多比较和交换log n次。pop_heap（下沉）：O(log n)，每次下沉需要比较两个子节点（2次比较），最多log n层。

Q10: 自定义类型的priority_queue需要注意什么？

A: 需要operator<或自定义比较器。比较器必须满足严格弱序（Strict Weak Ordering）：非自反性、非对称性、传递性、等价传递性。

11. 各容器如何选择

决策流程图



选择速查表

场景	推荐容器	原因
需要随机访问	vector	O(1)随机访问，缓存友好
尾插入+随机访问	vector	push_back O(1)摊销
头尾频繁插入删除	deque	push_front/pop_front都是O(1)
频繁中间插入删除	list	已知位置O(1)
key-value + 有序遍历	map	红黑树保证顺序
key-value + 快速查找	unordered_map	哈希表O(1)查找
有序去重	set	红黑树自动去重+排序
FIFO	queue	容器适配器

场景	推荐容器	原因
LIFO	stack	容器适配器
优先级处理	priority_queue	堆操作
编译期固定大小	array	栈上分配，零开销
字符串	string	SSO优化
视图/参数传递	string_view / span	非拥有，轻量

12. 完整对比表

容器全面对比

容器	底层数据结构	随机访问	插入(头)	插入(尾)	插入(中)	删除(头)	删除(尾)	删除(中)	查找	内存连续	缓存友好度
vector	动态数组	O(1)	O(n)	O(1)*	O(n)	O(n)	O(1)	O(n)	O(n)/O(log n)	是	最好
deque	分段数组	O(1)	O(1)	O(1)	O(n)	O(1)	O(1)	O(n)	O(n)	分段	较好
list	双向循环链表	不支持	O(1)	O(1)	O(1)	O(1)	O(1)	O(1)	O(n)	否	差
forward_list	单向链表	不支持	O(1)	O(1)**	O(1)**	O(1)	O(1)**	O(1)**	O(n)	否	差
map	红黑树	O(log n)***	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	否	差
set	红黑树	O(log n)***	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	否	差
unordered_map	哈希表(开链)	不支持	O(1)均	O(1)均	O(1)均	O(1)均	O(1)均	O(1)均	O(1)均	否	差
unordered_set	哈希表(开链)	不支持	O(1)均	O(1)均	O(1)均	O(1)均	O(1)均	O(1)均	O(1)均	否	差
string	动态char数组+SSO	O(1)	O(n)	O(1)*	O(n)	O(n)	O(1)	O(n)	O(n)	是	最好
queue	deque(默认)	不支持	-	O(1)	-	O(1)	-	-	-	-	-
stack	deque(默认)	不支持	-	O(1)	-	-	O(1)	-	-	-	-
priority_queue	vector+heap	不支持	-	O(log n)	-	-	O(log n)	-	-	-	-
array	固定数组	O(1)	-	-	-	-	-	-	O(n)	是	最好

- vector/string尾部的push_back是均摊O(1)，扩容时O(n)
- ** forward_list的尾部操作需要先遍历找到尾部，实际是O(n)
- *** map/set的随机访问指按key访问（不是按索引），按key是O(log n)但不支持按下标随机访问

迭代器失效规则

容器	插入	删除	其他失效
vector	扩容: 全部失效; 否则: 插入点之后	被删元素及之后	push_back可能触发扩容→全部失效

容器	插入	删除	其他失效
deque	中间: 全部失效; 头尾: 全部可能失效(但引用不失效)	中间: 全部失效; 头尾: 被删节点失效	map扩容→迭代器可能全部失效
list	不失效	仅被删元素	-
forward_list	不失效(insert_after)	仅被删元素	-
map/set	不失效	仅被删元素	-
unordered_map/set	rehash: 全部失效; 否则: 不失效	仅被删元素	-
string	同vector	同vector	同vector

空间开销对比

容器	每元素额外开销	说明
vector	0 (无额外指针)	但有capacity-size的预留空间
deque	buffer未用空间 + 中控器	buffer填充不饱满时有浪费
list	2 pointers = 16B	prev + next
forward_list	1 pointer = 8B	next only
map/multimap	3 ptr + color ~ 32B	parent, left, right, color
set/multiset	3 ptr + color ~ 32B	同上
unordered_map	1 ptr (~8B) + bucket数组	next指针 + 桶指针
unordered_set	1 ptr (~8B) + bucket数组	同上
string (long)	0 (同vector)	SSO模式下额外的15~22B本地buf
array	0	总大小 = sizeof(T) * N
stack/queue	同底层容器	适配器无额外开销

13. 专项深度剖析

13.1 vector扩容因子深度分析

数学推导：为什么扩容因子k=1.5可以内存复用？

设初始容量为c0，第i次扩容后容量 $c_i = c_0 * k^i$

第i次扩容后，释放的内存总量 $s_i = c_0 * (k^i - 1) / (k - 1)$

第i+1次扩容需要的容量 = $c_0 * k^{(i+1)}$

当 $s_i \geq c_0 * k^i$ （即释放的内存 $\geq$ 当前需要的）

即 $c_0 * (k^i - 1)/(k-1) \geq c_0 * k^i$

即 $(k^i - 1)/(k-1) \geq k^i$
即 $k^i - 1 \geq (k-1) \cdot k^i$
即 $k^i - 1 \geq k^{i+1} - k^i$
即 $2 \cdot k^i - k^{i+1} - 1 \geq 0$
即 $k^i \cdot (2 - k) \geq 1$

当 $k < 2$ 且 i 足够大时，不等式成立
当 $k = 1.5$ 时，需要 $i \geq \text{ceil}(\log_{1.5}(1/(2-1.5))) = 0$ ($i \geq 0$ 立即成立)
当 $k = 1.1$ 时， i 越大越容易满足，但小 i 时可能不满足
当 $k \geq 2$ 时，永远不满足

结论： $k < 2$ 才能使释放的内存最终可被后续分配复用
 $k = 1.5$ 是工程平衡点
 $k = 2$ 会导致内存永远不可复用

13.2 红黑树 vs 哈希表 深入对比

维度	红黑树 (map/set)	哈希表 (unordered_map/unordered_set)
查找复杂度	严格 $O(\log n)$	均摊 $O(1)$ ，最差 $O(n)$
插入复杂度	严格 $O(\log n)$	均摊 $O(1)$ ，最差 $O(n)$
内存效率	中等（每个节点有3个指针+color）	中等偏低（指针+bucket数组）
有序性	自然有序（按key升序）	无序
区间查询	$O(\log n)$ 支持	不支持
最值查询	$O(\log n)$	$O(n)$ （需遍历所有）
顺序遍历	直接中序遍历	需排序后遍历
迭代器失效	仅被删节点	rehash时全部失效
缓存友好	差（指针跟随）	差（指针跟随）
实现复杂度	高（旋转+染色）	中等（hash+链表）
对输入敏感	否	是（恶意输入可制造大量冲突）
使用场景举例	金融系统（需要稳定性）、字典	缓存、数据库索引

选择建议:

- 需要严格 $=O(\log n)$ 保证选红黑树（如金融交易系统）
- 需要 $O(1)$ 均摊且不关心顺序选哈希表
- 需要区间查询（查找key在[a,b]之间的元素）只能选红黑树
- 对内存要求严格时，数据量大选哈希表（负载因子可调），数据量小选红黑树

13.3 string SSO 深入分析

为什么GCC是15而Clang是22?

GCC (sizeof(string) = 32):

```
+-----+-----+
| local_buf[16] | capacity[8] |
+-----+-----+
| size[8]       |
+-----+-----+
```

本地buf = 16B, 需要1B存'\0'

最长SSO字符串 = 15字符

Clang (sizeof(string) = 24):

长模式:

```
+-----+-----+-----+
| ptr[8] | size[8] | cap[7] + |
+-----+-----+ flag[1] +
```

短模式:

```
+-----+-----+
| buf[22] | size |
+-----+ [1] +
```

Clang使用更紧凑的设计:

- 长模式: cap字段的bit0=1标记长模式, bit1-63存储capacity
- 短模式: 最后1字节的高4位存储size(0-22), 低4位存储特定掩码

本地buf = 22B, 需要1B存'\0'

最长SSO字符串 = 22字符

但size最大值只有22, 需要有方法表达23+ (用special sentinel)

为什么这种差异存在?

GCC: 简单直接的实现, 维护性好

Clang: 极致优化, 把24字节利用到极致

MSVC: 和GCC类似

哪个更好?

- LLVM的22字符: 覆盖了更多常见短字符串, 减少了堆分配
- GCC的15字符: 实现更简单, 代码更易维护

实际benchmark中Clang的string通常比GCC略快

13.4 deque buffer大小计算

```
// GCC中deque buffer大小的确定逻辑
inline size_t
__deque_buf_size(size_t __size) {
    // __size = sizeof(T)
    return __size < 512
        ? size_t(512 / __size) // buffer = 512字节
        : size_t(1);           // 大对象: buffer只存1个元素
}

// 示例:
// sizeof(int) = 4  => buffer_size = 512/4 = 128个int/buffer
// sizeof(double) = 8 => buffer_size = 512/8 = 64个double/buffer
// sizeof(large_struct) = 4096 => buffer_size = 1
```

13.5 priority_queue实现选择对比

优先队列的五种实现:

实现方式	push	pop	top
无序数组	$O(1)$	$O(n)$	$O(n)$
有序数组	$O(n)$	$O(1)$	$O(1)$
无序链表	$O(1)$	$O(n)$	$O(n)$
有序链表	$O(n)$	$O(1)$	$O(1)$
二叉堆	$O(\log n)$	$O(\log n)$	$O(1)$

二叉堆是唯一在push和pop上都达到 $O(\log n)$ 的选择

加上用vector存储 = 零指针开销 + 缓存友好

这就是为什么STL选它

额外好处: make_heap是 $O(n)$ 线性时间建堆

sort_heap是 $O(n \log n)$ 原地堆排序

14. 面试考点总结

14.1 面试重点 (必须掌握)

1. **vector扩容机制**: 扩容因子 (1.5x vs 2x)、为什么、扩容对迭代器的影响
2. **deque的分段设计**: map+buffer、为什么需要、和vector的对比
3. **红黑树原理**: 5条性质、为什么选它 (vs AVL/哈希表)、map/set共用
4. **哈希表**: 开链法、rehash、负载因子、hash函数设计
5. **SSO**: 短字符串优化、不同实现的阈值、为什么需要
6. **迭代器失效**: 每种容器在哪些操作后迭代器失效
7. **容器适配器**: stack/queue/priority_queue的底层容器及理由

8. 容器选择：给定场景选最合适的容器

14.2 高频考点

考点	出现频率	难度
vector扩容因子为什么是1.5或2	极高	中等
vector的push_back均摊O(1)分析	极高	中等
map和unordered_map对比	极高	简单
红黑树 vs AVL树	高	中等
deque底层结构	高	中等
SSO原理	高	中等
迭代器失效规则	高	中等
priority_queue底层	中	简单
COW为什么被废弃	中	中等
vector的陷阱	中	简单

14.3 常见陷阱

1. **vector的push_back不总为O(1)**：扩容时是O(n)，但均摊是O(1)
2. **clear()不释放内存**：capacity()不变，需要 `v.shrink_to_fit()` 或 `swap` 技巧
3. **vector是bitset**：不是标准容器，不返回bool&，遍历要小心
4. **迭代器循环中erase**：

```
for (auto it = v.begin(); it != v.end(); ) { if (cond) it = v.erase(it); else ++it; }
```
5. **map的key是const**：即使通过iterator也不能修改first
6. **unordered_map的rehash**：会使所有迭代器失效
7. **string的c_str()生命周期**：string对象销毁后c_str()返回的指针悬垂
8. **operator[] vs at()**：前者无越界检查（UB），后者抛异常
9. **list::size()曾是O(n)**：C++11前GCC中是O(n)，现已修复
10. **reserve vs resize**：reserve改capacity不改size；resize改size可能插入默认值

14.4 建议背诵内容

必背代码片段1：vector扩容原理

```
// vector扩容的核心：分配更大的内存，移动/拷贝旧元素
// 扩容大小：size + max(size, n)
// GCC：2倍扩容，MSVC：1.5倍扩容

// 减少扩容只此一招：
v.reserve(expected_size);
```

必背代码片段2：swap技巧释放内存

```
// 释放vector的多余容量
vector<int>().swap(v);           // 清空并释放
vector<int>(v).swap(v);         // 收缩到刚好容纳现有元素(shrink to fit)
// C++11推荐：
v.shrink_to_fit();              // 非强制，但通常有效
v.clear(); v.shrink_to_fit();    // 清空并释放
```

必背代码片段3：正确的循环删除

```
// vector/list/set：通用方法
for (auto it = container.begin(); it != container.end(); ) {
    if (should_remove(*it)) {
        it = container.erase(it); // 返回下一个有效迭代器
    } else {
        ++it;
    }
}
// C++20: std::erase_if
std::erase_if(container, [](const auto& elem) { return should_remove(elem); });
```

必背代码片段4：红黑树5条性质

1. 节点是红或黑
2. 根是黑
3. 所有叶子(NIL)是黑
4. 红节点的子节点都是黑（无连续红）
5. 任意节点到其每个叶子路径包含相同数量黑节点

必背代码片段5：常见容器的底层结构速记

```
vector   = 动态数组
list     = 双向循环链表(带头结点)
deque    = 分段连续空间(map+多buffer)
map/set  = 红黑树
unordered_map/set = 哈希表(开链法)
string   = 动态char数组 + SSO
queue/stack = deque(默认)适配
priority_queue = vector + 二叉最大堆
```

15. 终极大对比表

容器	数据结构	元素访问	头部 insert	尾部 insert	中间 insert	头部 erase	尾部 erase	中间 erase	查找	排序	迭代器类型	迭代器失效(insert)	迭代器失效(erase)	内存连续性	预留空间
vector	动态数组	O(1) 随机	O(n)	O(1)*	O(n)	O(n)	O(1)	O(n)	O(n)**	O(n log n)	随机访问	扩容全部/插入后	删除后	是	reserve
deque	分段连续	O(1) 随机	O(1)	O(1)	O(n)	O(1)	O(1)	O(n)	O(n)	O(n log n)	随机访问	全部可能失效	删除后全部可能	分段	无
list	双向链表	O(n)	O(1)	O(1)	O(1)	O(1)	O(1)	O(1)	O(n)	O(n log n) (归并)	双向	不失效	被删元素	否	无
forward_list	单向链表	O(n)	O(1)	O(1)†	O(1)†	O(1)	O(1)†	O(1)†	O(n)	O(n log n)	前向	不失效	被删元素	否	无
map	红黑树	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	自然有序	双向	不失效	被删元素	否	无
multimap	红黑树	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	自然有序	双向	不失效	被删元素	否	无
set	红黑树	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	自然有序	双向	不失效	被删元素	否	无
multiset	红黑树	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	自然有序	双向	不失效	被删元素	否	无
unordered_map	哈希表	O(1)均	O(1)均	O(1)均	O(1)均	O(1)均	O(1)均	O(1)均	O(1)均	无序	前向	rehash全部	被删元素	否	reserve
unordered_set	哈希表	O(1)均	O(1)均	O(1)均	O(1)均	O(1)均	O(1)均	O(1)均	O(1)均	无序	前向	rehash全部	被删元素	否	reserve
string	动态 char+SSO	O(1) 随机	O(n)	O(1)*	O(n)	O(n)	O(1)	O(n)	O(n)	O(n log n)	随机访问	同vector	同vector	是	reserve
array	固定数组	O(1) 随机	N/A	N/A	N/A	N/A	N/A	N/A	O(n)	O(n log n)	随机访问	N/A	N/A	是	编译期固定
stack	deque	仅top	N/A	O(1)	N/A	N/A	O(1)	N/A	N/A	N/A	无	同底层	同底层	同底层	同底层
queue	deque	仅 front/back	N/A	O(1)	N/A	O(1)	N/A	N/A	N/A	N/A	无	同底层	同底层	同底层	同底层
priority_queue	vector+heap	仅top	N/A	O(log n)	N/A	N/A	O(log n)	N/A	N/A	N/A	无	同底层	同底层	同底层	同底层

- 均摊O(1)
- ** 有序时可用二分查找O(log n)
- † forward_list的尾部操作需要遍历到尾部，实际是O(n)

本章小结

STL容器的源码分析是C++面试中最核心、最深入的部分。掌握本章内容需要做到：

1. **理解底层**：每种容器的数据结构不是凭空设计的，都有其取舍
2. **记住复杂度**：时间复杂度表是面试必问项，需要肌肉记忆
3. **知道为什么**：设计理由比裸记更重要，面试官看重思考过程
4. **能画图**：内存布局图能帮助你面试官建立共同理解
5. **会选容器**：给定场景选最合适的容器是最终目标

学习建议：每学完一个容器，尝试回答3个终极问题：

- 为什么这样设计？（设计哲学）
- 有没有更好的选择？（批判性思维）
- 在实际项目中什么时候用？（工程角度）

能够清晰回答这些问题，说明已经真正掌握了STL源码思想。